CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL – UDF

COORDENAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO

Anna Beatriz Resende Oki Corrêa

Protótipo para converter fórmulas de planilha com condicionais aninhadas em Python

BRASÍLIA

2026

Anna Beatriz Resende Oki Corrêa

Protótipo para converter fórmulas de planilha com condicionais aninhadas em Python

Trabalho de conclusão de curso apresentado à Coordenação do Curso de Sistemas de Informação, do Centro Universitário do Distrito Federal - UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em Sistemas de Informação.

Orientador: Prof. Dr. Xxxx

BRASÍLIA

?Sobrenome?, ?Nome?. Título : ?author?. – Brasília, 2026. 27 f.

Orientador: ?orientador?. Trabalho de Conclusão de Curso (Graduação – Sistemas de Informação) – Centro Universitário do Distrito Federal – UDF. Coordenação do Curso de Sistemas de Informação, Brasília, DF, 2026.

1. ?Assunto?. 2. ?Assunto?. 3. ?Assunto?. I. ?Título?.

CDU: ?consultar na biblioteca?

?Titulo?

Trabalho de conclusão de curso apresentado à ?Coordenação?, do Centro Universitário do Distrito Federal – UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em ?Curso?.

Orientador: ?orientador?.

Brasília, _____ de _____ de 2026.

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

Nota: _____

Dedico este trabalho à minha família, pelo apoio, incentivo e compreensão durante toda a minha trajetória acadêmica.

Agradecimentos

Agradeço, primeiramente, a Deus, por mais uma conquista; ao meu orientador, pela dedicação e correções; e aos bibliotecários pelo suporte em todas as pesquisas.

"A educação é a arma mais poderosa que você pode usar
para mudar o mundo."

Nelson Mandela

Resumo

Profissionais de tecnologia precisam, com frequência, passar uma mesma regra de uma representação para outra, como de uma fórmula de planilha para um código. Quando a fórmula tem condições aninhadas, a lógica fica escondida e é fácil esquecer uma condição. Este trabalho propõe um protótipo que converte fórmulas de planilha com condições aninhadas em código Python e mostra a lógica em forma de fluxograma. A lógica da fórmula é organizada em uma árvore de condições, da qual saem o código e o fluxograma. A avaliação compara o resultado da fórmula com o do código em casos de teste e reúne a opinião de um profissional sobre a utilidade do fluxograma. O trabalho está em andamento: o problema, a proposta e o plano de avaliação estão definidos, e o desenvolvimento do protótipo será a próxima etapa.

Palavras-chave: *lógica computacional; fórmulas de planilha; conversão de código; fluxograma; Python.*

Abstract

Technology professionals often need to move the same rule from one representation to another, such as from a spreadsheet formula to code. When a formula has nested conditions, the logic is hidden and it is easy to miss a condition. This work proposes a prototype that converts spreadsheet formulas with nested conditions into Python code and shows the logic as a flowchart. The formula's logic is organized as a tree of conditions, from which the code and the flowchart are generated. The evaluation compares the formula's result with the code's result on test cases and collects a professional's opinion on the usefulness of the flowchart. The work is in progress: the problem, the proposal and the evaluation plan are defined, and the prototype development is the next step.

Keywords: *computational logic; spreadsheet formulas; code conversion; flowchart; Python.*

Lista de ilustrações

1 Processo de abstração aplicado à Ciência da Computação. 16
2 Relação entre diferentes formas de representação da lógica computacional. . . . 20
3 Exemplo de lógica computacional representada por meio de um sistema visual baseado em nós (blueprint). 21

Siglas

Sumário

1 INTRODUÇÃO 13

2 DESENVOLVIMENTO 16
2.1 Abstração e Representação da Lógica Computacional 16
2.1.1 O conceito de abstração em computação 16
2.1.2 Lógica computacional independente da linguagem 17
2.1.3 Representações da lógica computacional 18
2.2 Programação Visual e Ambientes Baseados em Fluxos 18
2.2.1 Conceitos de programação visual 18
2.2.2 Fluxogramas e representação de algoritmos 19
2.2.3 Blueprints e sistemas baseados em nós 20
2.2.4 Plataformas Low-Code e No-Code 21

3 PROPOSTA CONCEITUAL DA PESQUISA 23
3.1 O problema 23
3.2 Visão geral da solução 23
3.3 Como o protótipo funciona 23
3.4 Escopo 24
3.5 Plano de avaliação 24
3.6 Limitações 25
3.7 Situação atual 25

4 CONSIDERAÇÕES FINAIS 26

REFERÊNCIAS 27

1 Introdução

A lógica computacional é a base do desenvolvimento de software. Ela define as regras, as decisões e as condições que fazem um sistema se comportar de determinada forma. Para ser usada, essa lógica precisa ser escrita de algum jeito que o computador consiga executar.

A abstração ajuda a lidar com a complexidade da computação. Com ela, deixam-se de lado os detalhes de uma implementação para olhar só o que importa em um problema (ABELSON; SUSSMAN, 1996). Por isso, uma mesma lógica pode ser escrita de formas diferentes. Uma condição, por exemplo, pode aparecer como um `if` em uma linguagem de programação, como um losango em um fluxograma ou como um nó em um sistema visual. A forma muda, mas o comportamento esperado continua o mesmo. Neste trabalho, chama-se de **lógica** o conjunto de regras e decisões, e de **representação** a forma usada para escrevê-las. A programação visual, que usa elementos gráficos para construir programas, é um exemplo de representação (MYERS, 1990; NOONE; MOONEY, 2018).

Na prática, profissionais de tecnologia precisam passar uma mesma lógica de uma representação para outra. Isso acontece, por exemplo, quando uma regra escrita em uma fórmula de planilha precisa virar um código, ou o contrário. Nessa passagem, a lógica não pode mudar. Se uma condição for esquecida, o resultado fica errado.

Esse problema apareceu em um questionário exploratório aplicado a quatro profissionais de Tecnologia da Informação. Um deles, que usa programação como apoio técnico no trabalho, relatou que precisa levar fórmulas de planilha para Python, e também o caminho inverso, mantendo o mesmo resultado. A maior dificuldade são as fórmulas com condições aninhadas, isto é, um `SE` dentro de outro `SE`. Nelas, a lógica fica escondida na sintaxe e é fácil deixar passar uma condição. Hoje ele faz esse trabalho à mão: lista as condições, reescreve a solução e testa. Segundo ele, isso gera retrabalho, e a perda de tempo é a consequência mais importante. Dois outros participantes relataram dificuldades parecidas ao adaptar uma lógica entre tecnologias diferentes. Como foram só quatro respostas, o questionário mostra que a necessidade existe, mas não permite dizer que todos os profissionais passam por isso.

A literatura mostra que passar código de uma linguagem para outra é uma tarefa difícil, mesmo para modelos de Inteligência Artificial. Pan et al. (2024) avaliaram a tradução de código entre cinco linguagens e consideraram uma tradução correta apenas quando o código compila, executa e passa nos testes. Nesse estudo, o percentual de traduções corretas ficou entre 2,1% e 47,3%, conforme o modelo. Isso mostra que gerar um código que parece certo não basta: é preciso verificar se ele se comporta como o original. Nos trabalhos consultados até agora, não foi encontrado

um estudo que trate especificamente da conversão de fórmulas de planilha com condições aninhadas para código.

Diante disso, o **problema** deste trabalho é a dificuldade de converter fórmulas de planilha com condições aninhadas em código sem perder nenhuma condição, e de conferir se o resultado continua igual.

O **objetivo geral** é desenvolver e avaliar um protótipo que converte fórmulas de planilha com condições aninhadas em código Python e mostra a lógica em forma de fluxograma. Os **objetivos específicos** são:

- definir quais tipos de fórmula o protótipo vai aceitar;
- representar a lógica da fórmula em uma estrutura independente da forma em que ela foi escrita;
- gerar, a partir dessa estrutura, o código Python e o fluxograma;
- verificar se o código gerado dá o mesmo resultado da fórmula em casos de teste;
- avaliar, com um profissional da área, se o fluxograma ajuda a entender a lógica.

A **solução proposta** é uma ferramenta pequena. Ela recebe uma fórmula, identifica suas condições e organiza essa lógica em uma estrutura em forma de árvore. A partir dessa árvore, a ferramenta gera o código Python e o fluxograma. Como os dois saem da mesma estrutura, a lógica é a mesma nas três formas: fórmula, código e fluxograma. Essa estrutura faz o papel de representação intermediária, ideia que vem dos estudos sobre abstração e representação da lógica. O uso de regras fixas ou de Inteligência Artificial para fazer a conversão será definido durante o desenvolvimento.

O público a que se destina são profissionais que usam planilhas e códigos no trabalho e precisam passar regras de uma forma para a outra. O profissional que relatou o problema será o principal usuário na avaliação.

Do ponto de vista metodológico, este trabalho é uma pesquisa aplicada, com desenvolvimento de um protótipo (GIL, 2002; PRODANOV; FREITAS, 2013). As etapas são: pesquisa bibliográfica, levantamento de fórmulas reais e de exemplo, desenvolvimento do protótipo e avaliação. A avaliação terá duas partes: testes que comparam o resultado da fórmula com o do código gerado, e a opinião do profissional sobre o fluxograma. Ficam fora do escopo outras linguagens, programação por blocos e uma aplicação web completa.

Este trabalho está dividido em quatro capítulos. Esta introdução apresenta o contexto, o problema, os objetivos e a metodologia. O Capítulo 2 apresenta a base teórica sobre abstração,

representações da lógica e programação visual. O Capítulo 3 apresenta a proposta do protótipo e o plano de avaliação. O Capítulo 4 traz as considerações finais.

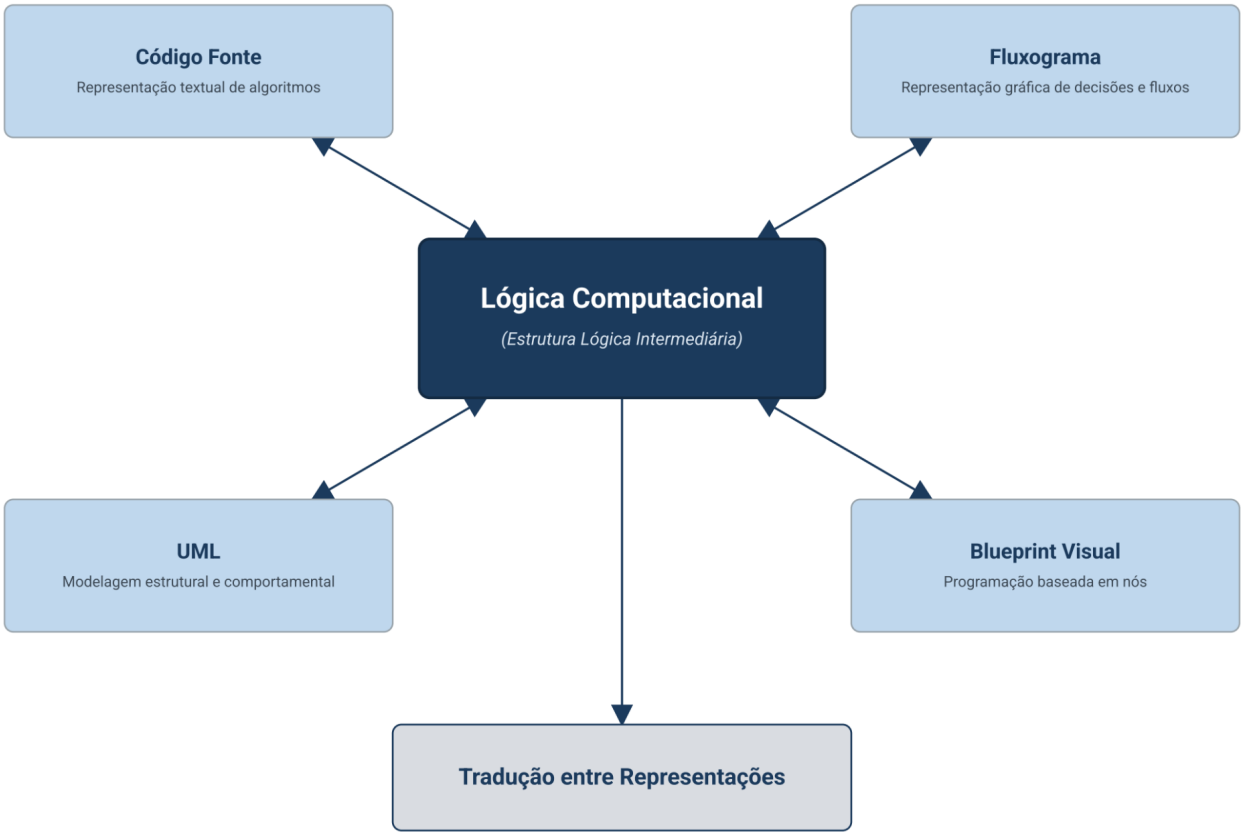
2 Desenvolvimento

2.1 ABSTRAÇÃO E REPRESENTAÇÃO DA LÓGICA COMPUTACIONAL

2.1.1 O conceito de abstração em computação

A abstração é considerada um dos conceitos fundamentais da Ciência da Computação, sendo utilizada para representar sistemas complexos por meio de modelos simplificados que destacam apenas os aspectos relevantes para determinado contexto. Segundo Abelson e Sussman (1996), a abstração permite que problemas computacionais sejam compreendidos e manipulados em diferentes níveis, reduzindo a necessidade de lidar diretamente com todos os detalhes de implementação, conforme a Figura 1.

Figura 1 – Processo de abstração aplicado à Ciência da Computação.



Fonte: elaboração do autor com base em Abelson e Sussman (1996).

No desenvolvimento de software, a abstração está presente em diversas camadas, desde

as linguagens de programação até as interfaces utilizadas pelos usuários finais. Cada camada oculta parte da complexidade da camada inferior, permitindo que desenvolvedores e usuários interajam com sistemas computacionais de maneira mais eficiente (ABELSON; SUSSMAN, 1996). Dessa forma, uma mesma lógica pode ser representada por diferentes mecanismos sem que seu significado fundamental seja alterado.

A compreensão desse conceito é especialmente relevante para esta pesquisa, uma vez que a hipótese investigada parte da possibilidade de que diferentes representações computacionais, como código textual e *blueprints* visuais, possam expressar uma mesma estrutura lógica subjacente. Nesse contexto, a abstração torna-se um elemento central para compreender como a lógica computacional pode ser representada e interpretada em diferentes formatos.

2.1.2 Lógica computacional independente da linguagem

A lógica computacional pode ser compreendida como um conjunto de regras, decisões e processos que definem o comportamento de um sistema independentemente da linguagem utilizada para implementá-lo. Nesse contexto, as linguagens de programação atuam como mecanismos de representação dessa lógica, fornecendo estruturas sintáticas capazes de expressar algoritmos e comportamentos computacionais (ABELSON; SUSSMAN, 1996).

Segundo Knuth (1968), os algoritmos constituem a base da computação, sendo independentes da tecnologia ou linguagem utilizada para sua implementação. Dessa forma, uma mesma solução lógica pode ser representada em diferentes linguagens de programação sem que seu objetivo fundamental seja alterado. O que muda é a forma de representação, enquanto a lógica subjacente permanece preservada.

Embora Knuth (1968) trate especialmente da independência entre algoritmos e linguagens textuais, esse mesmo princípio pode ser estendido às representações visuais. Uma decisão condicional, por exemplo, pode ser descrita por comandos em uma linguagem textual ou por elementos gráficos em fluxogramas e *blueprints*, sem que a relação lógica entre a condição e seus possíveis fluxos de execução seja modificada. Dessa forma, a independência entre lógica e representação não se limita às linguagens de programação, estendendo-se também às formas visuais de representação computacional.

Essa independência entre lógica e representação, sustentada pelos conceitos de abstração de Abelson e Sussman (1996) e pela noção de algoritmo de Knuth (1968), constitui um dos fundamentos teóricos para investigar a viabilidade de traduções entre código textual e representações visuais. Compreender a lógica como uma estrutura separada de sua forma de representação é, portanto, condição necessária para analisar as relações existentes entre os diferentes modelos abordados nas seções seguintes.

2.1.3 Representações da lógica computacional

A lógica computacional pode ser expressa por diferentes modelos de representação, que variam conforme o contexto, o nível de abstração e os objetivos pretendidos. Ao longo da evolução da computação, essas formas foram desenvolvidas para atender a finalidades distintas, como facilitar a compreensão, documentar processos ou implementar sistemas executáveis.

Essas representações podem ser organizadas segundo diferentes critérios. Quanto à forma, distinguem-se as representações textuais (como o pseudocódigo e as linguagens de programação) das representações visuais (como os fluxogramas, os diagramas UML e os ambientes de programação baseados em nós). Quanto à finalidade, algumas representações possuem caráter essencialmente descritivo ou documental, voltado à compreensão e à comunicação de algoritmos, enquanto outras são executáveis, capazes de produzir diretamente o comportamento que descrevem. Apesar dessas diferenças, todas compartilham o objetivo comum de representar comportamentos, decisões e processos computacionais.

Essa diversidade de representações foi sistematizada por Myers (1990), que propôs taxonomias para a programação visual e a visualização de programas, evidenciando que elementos gráficos podem representar relações e fluxos lógicos sem substituir a lógica computacional subjacente. Nessa perspectiva, a representação visual não se opõe ao código textual, mas oferece uma forma alternativa de expressar a mesma estrutura lógica.

A coexistência de múltiplas representações para uma mesma lógica sugere a existência de uma estrutura comum que as conecta. Compreender como cada modelo organiza e expressa algoritmos é, assim, um passo necessário para examinar as relações entre representações textuais e visuais, tema aprofundado na seção seguinte, dedicada à programação visual e aos ambientes baseados em fluxos.

2.2 PROGRAMAÇÃO VISUAL E AMBIENTES BASEADOS EM FLUXOS

2.2.1 Conceitos de programação visual

A programação visual é uma abordagem de desenvolvimento na qual a lógica computacional é representada por elementos gráficos, como blocos, nós e conexões, em vez de ser expressa exclusivamente por código textual. Essa forma de representação busca tornar a construção e compreensão de sistemas mais intuitiva, permitindo que processos e comportamentos sejam visualizados de maneira direta.

Conforme Myers (1990), ambientes de programação visual foram desenvolvidos para

facilitar a interação com sistemas computacionais por meio de representações gráficas da lógica. Embora utilizem elementos visuais, esses ambientes continuam baseados nos mesmos princípios computacionais encontrados nas linguagens tradicionais de programação. A literatura também discute o papel das linguagens visuais e textuais, sobretudo na aprendizagem inicial de programação (NOONE; MOONEY, 2018).

Essa característica revela um aspecto central da programação visual: por se apoiar nos mesmos princípios computacionais das linguagens textuais, ela demonstra que um mesmo comportamento pode ser implementado tanto por código quanto por fluxos visuais. Essa equivalência é o que torna os ambientes visuais especialmente relevantes para esta pesquisa, e as próximas subseções examinam suas principais manifestações — os fluxogramas, os blueprints e as plataformas de baixo código.

2.2.2 Fluxogramas e representação de algoritmos

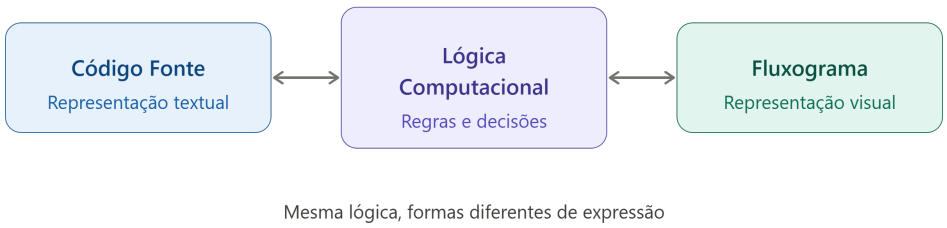
Os fluxogramas constituem uma das formas mais tradicionais de representação visual de algoritmos. Por meio de símbolos padronizados e conexões direcionais, eles permitem representar decisões, processos e fluxos de execução de maneira gráfica, facilitando a compreensão da lógica computacional.

Além de sua função representativa, os fluxogramas são amplamente utilizados para documentar e comunicar algoritmos durante o desenvolvimento de software. Sua estrutura visual torna mais acessível a compreensão do comportamento de sistemas computacionais, especialmente em etapas de análise, planejamento e documentação.

Cabe destacar, no entanto, que os fluxogramas possuem caráter essencialmente descritivo. Eles representam a lógica de um algoritmo para fins de compreensão e comunicação, mas não constituem, por si sós, um mecanismo de execução: a lógica representada precisa ser posteriormente traduzida em código para ser executada por um sistema computacional. Essa distinção entre representar e executar é relevante para esta pesquisa, pois diferencia os fluxogramas de outras representações visuais, como os blueprints, capazes de executar diretamente a lógica que descrevem, conforme discutido na seção seguinte.

A Figura 2 apresenta a lógica computacional como elemento central entre diferentes formas de representação. Nesse contexto, tanto o código-fonte quanto os fluxogramas podem expressar uma mesma estrutura lógica, diferenciando-se apenas pela forma como essa informação é apresentada ao desenvolvedor.

Figura 2 – Relação entre diferentes formas de representação da lógica computacional.



Fonte: elaboração do autor com base em Abelson e Sussman (1996) e Knuth (1968).

2.2.3 Blueprints e sistemas baseados em nós

Os *blueprints* e sistemas baseados em nós representam uma evolução dos modelos tradicionais de programação visual (ROMERO, 2014). Nesses ambientes, a lógica computacional é construída por meio da conexão entre elementos gráficos denominados nós, que representam operações, eventos, decisões ou fluxos de dados.

Diferentemente dos fluxogramas, que possuem principalmente função descritiva e documental, os sistemas baseados em nós permitem a execução direta da lógica construída visualmente. Dessa forma, o fluxo visual deixa de ser apenas uma representação e passa a atuar como mecanismo efetivo de desenvolvimento de software.

Um dos exemplos mais conhecidos dessa abordagem é o sistema Blueprint da Unreal Engine, utilizado para o desenvolvimento de jogos e aplicações interativas. Nesse ambiente, programadores e designers podem construir comportamentos complexos por meio da conexão visual de nós, reduzindo a necessidade de escrita manual de código textual.

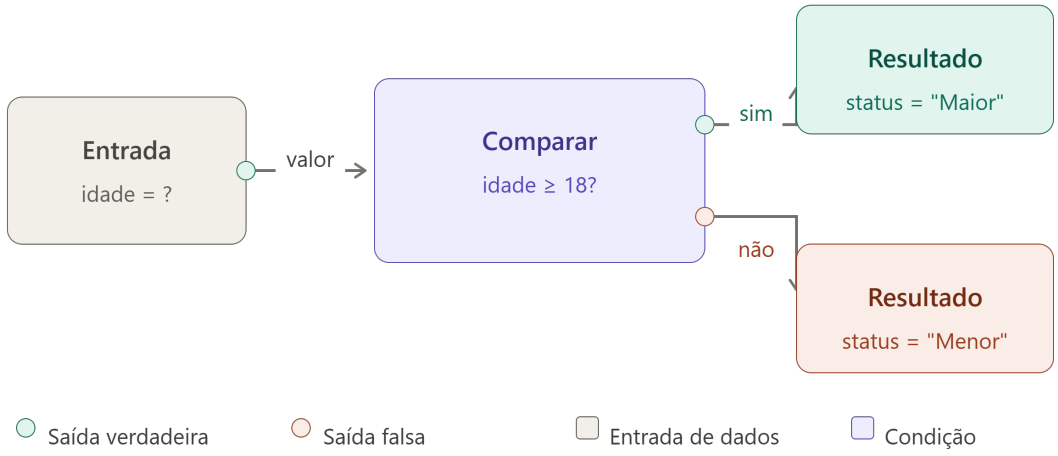
Essa característica reforça a hipótese de que a lógica computacional pode ser separada de sua forma de representação. Embora os *blueprints* utilizem elementos visuais, os comportamentos implementados seguem os mesmos princípios algorítmicos encontrados nas linguagens de programação tradicionais.

A documentação oficial da Epic Games (Epic Games, 2025) descreve o sistema Blueprint como uma interface de scripting visual que disponibiliza recursos de programação por meio de nós conectados. A solidez lógica dessa abordagem é evidenciada por trabalhos que demonstram ser possível aplicar verificação formal sobre scripts construídos em blueprints. Wayama et al. (2022), por exemplo, aplicaram model checking a programas desenvolvidos no Blueprint da Unreal Engine 5, evidenciando que a lógica representada visualmente pode ser formalmente analisada. Isso re-

força a ideia central desta pesquisa: a de que a representação visual expressa uma estrutura lógica computacional bem definida, e não apenas um recurso gráfico de apoio.

Nesse contexto, os sistemas baseados em nós demonstram que diferentes formas de representação podem expressar uma mesma estrutura lógica, aspecto que se relaciona diretamente com a proposta desta pesquisa. A Figura 3 ilustra esse conceito por meio de um exemplo lógico simples — a condição `if idade >= 18` —, desta vez expressa no formato de nós conectados característico dos sistemas blueprint. Embora a representação seja visualmente distinta do código textual e do fluxograma, o comportamento lógico subjacente permanece idêntico.

Figura 3 – Exemplo de lógica computacional representada por meio de um sistema visual baseado em nós (blueprint).



Fonte: elaboração do autor com base em Romero (2014) e Epic Games (2025).

2.2.4 Plataformas Low-Code e No-Code

As plataformas *Low-Code* e *No-Code* surgiram como uma evolução dos ambientes de programação visual, buscando reduzir a complexidade do desenvolvimento de software por meio de interfaces gráficas e componentes reutilizáveis. Nessas plataformas, grande parte da lógica é construída por meio da conexão entre blocos, fluxos e elementos visuais, diminuindo a necessidade de escrita manual de código.

Há indícios, em determinados contextos, de que ambientes de programação visual podem favorecer a aprendizagem e a compreensão de conceitos de programação. Sáez-López, Román-González e Vázquez-Cano (2016), em um estudo de caso quase-experimental com 107 estudantes do ensino fundamental que utilizaram o Scratch, relatam resultados relacionados à

aprendizagem de conceitos de programação, de lógica e de práticas computacionais. Esses resultados referem-se a um contexto educacional específico e não permitem concluir que a programação visual seja superior à textual.

Por sua vez, plataformas como *n8n*, *Node-RED*, *Microsoft Power Automate* e outras ferramentas de automação utilizam fluxos visuais para representar processos, integrações e comportamentos computacionais. Embora apresentem interfaces simplificadas, essas plataformas continuam baseadas nos mesmos princípios lógicos encontrados nas linguagens tradicionais de programação (SILVA et al., 2023).

Cada uma dessas ferramentas adota uma forma própria de representar a lógica, ainda que compartilhem o princípio comum dos fluxos visuais. No *n8n*, a lógica é organizada em nós que atuam como blocos de construção de um fluxo de trabalho, responsáveis por iniciar a execução, buscar e enviar dados e manipular informações (n8n, 2026). O *Node-RED*, por sua vez, baseia-se na programação orientada a eventos e no encadeamento de nós, sendo amplamente aplicado à integração de dispositivos e à Internet das Coisas (OpenJS Foundation, 2026). Já o *Microsoft Power Automate* estrutura a lógica em fluxos compostos por gatilhos, que iniciam a execução a partir de um evento, e ações, que definem as operações realizadas em seguida (Microsoft, 2026). Apesar das diferenças visuais e estruturais entre essas representações, todas expressam decisões, condições e sequências de execução, evidenciando que uma mesma lógica computacional pode assumir formas distintas conforme o ambiente utilizado.

Síntese da seção

A análise dos ambientes apresentados demonstra que a lógica computacional pode ser representada por diferentes modelos visuais (fluxogramas, blueprints e plataformas *Low-Code* ou *No-Code*) sem perder sua capacidade de expressar algoritmos, decisões e fluxos de execução. Embora cada abordagem possua características e objetivos distintos, todas compartilham um princípio comum: a utilização de elementos visuais para representar estruturas lógicas, sem alteração de seu significado fundamental.

Essa constatação fornece suporte teórico à hipótese investigada nesta pesquisa e estabelece a base para o capítulo seguinte, no qual se discute como essa estrutura lógica comum pode atuar como elemento intermediário na tradução entre diferentes representações computacionais.

3 Proposta Conceitual da Pesquisa

3.1 O PROBLEMA

Em planilhas, é comum escrever regras de negócio por meio de fórmulas. Quando a regra tem várias situações possíveis, a fórmula passa a ter condições dentro de condições. Essas fórmulas são difíceis de ler, porque a lógica fica escondida na sintaxe. Ao levar a regra para um código, como o Python, é fácil esquecer uma das condições.

Hoje, esse trabalho é feito à mão: a pessoa lista as condições, reescreve a solução e testa alguns casos. Isso toma tempo e depende de atenção. Esse relato vem do questionário exploratório descrito na introdução e, por isso, indica uma necessidade real, sem permitir generalizações.

3.2 VISÃO GERAL DA SOLUÇÃO

A solução proposta é um protótipo que recebe uma fórmula de planilha com condições aninhadas e entrega três coisas:

- o código Python equivalente;
- um fluxograma com a lógica da fórmula;
- um relatório de testes que compara o resultado da fórmula com o do código.

O principal usuário é o profissional que precisa passar regras entre planilha e código. O protótipo deve ser simples: não é preciso uma aplicação web completa, e basta que ele seja usado em casos reais de pequeno porte.

3.3 COMO O PROTÓTIPO FUNCIONA

O funcionamento tem quatro etapas:

1. **Leitura:** a fórmula é lida e suas partes são identificadas (condições, valores e resultados).
2. **Árvore de condições:** a lógica é organizada em uma árvore, em que cada nó é uma decisão e cada caminho leva a um resultado.

3. **Geração:** da mesma árvore saem o código Python e o fluxograma.
4. **Verificação:** os mesmos casos de teste são aplicados à fórmula e ao código, e os resultados são comparados.

A árvore de condições é o centro da proposta. Ela é a representação intermediária: não depende da sintaxe da planilha nem da do Python. Por isso, os dois resultados saem da mesma lógica, o que reduz o risco de uma condição aparecer em um e faltar no outro. Essa escolha segue a ideia de abstração discutida no Capítulo 2 (ABELSON; SUSSMAN, 1996).

3.4 ESCOPO

Na primeira versão, o protótipo deve aceitar fórmulas com:

- condições do tipo `SE`, aninhadas;
- comparações simples (maior, menor, igual);
- combinações com `E` e `OU`;
- resultados que sejam números ou textos.

Ficam fora do escopo: outras linguagens de destino, programação por blocos, *blueprints* e uma aplicação web completa. O uso de regras fixas ou de Inteligência Artificial para a conversão será decidido durante o desenvolvimento, conforme a viabilidade e os resultados dos testes.

3.5 PLANO DE AVALIAÇÃO

A avaliação terá duas partes.

1. **Correção:** fórmulas reais, fornecidas pelo profissional (sem dados sigilosos), e fórmulas de exemplo criadas pela autora serão convertidas. Para cada uma, será montado um conjunto de casos de teste. O código é considerado correto quando dá o mesmo resultado da fórmula em todos os casos. Gerar um código que apenas roda sem erro não é suficiente.
2. **Utilidade do fluxograma:** o profissional que relatou o problema vai analisar os fluxogramas e dizer se eles ajudam a entender a lógica e a encontrar condições esquecidas.

Também serão registrados os erros e as fórmulas que o protótipo não consegue converter, para discutir seus limites.

3.6 LIMITAÇÕES

O trabalho se limita a um tipo de problema e a um usuário principal, e os resultados não podem ser generalizados para todos os profissionais. A quantidade de fórmulas avaliadas será pequena. Além disso, o protótipo não cobre todas as funções de planilha. Essas limitações serão discutidas junto com os resultados.

3.7 SITUAÇÃO ATUAL

Até o momento, o problema, a solução e o plano de avaliação foram definidos. O protótipo, o levantamento das fórmulas reais e a avaliação ainda serão realizados.

4 Considerações Finais

Este trabalho parte de um problema concreto: converter fórmulas de planilha com condições aninhadas em código sem perder nenhuma condição. Esse problema foi relatado por um profissional de tecnologia em um questionário exploratório.

Para tratar esse problema, foi proposto um protótipo que organiza a lógica da fórmula em uma árvore de condições e, a partir dela, gera o código Python e um fluxograma. Essa proposta usa o que foi estudado sobre abstração e representação da lógica: a mesma lógica pode ser escrita de formas diferentes sem mudar o comportamento.

Até aqui, foram definidos o problema, os objetivos, o escopo e o plano de avaliação. O protótipo ainda não foi desenvolvido, e por isso ainda não há resultados.

Os próximos passos são: reunir fórmulas reais e de exemplo, desenvolver o protótipo, aplicar os testes de correção e coletar a opinião do profissional sobre os fluxogramas. Com esses resultados, será possível discutir o que funcionou, quais fórmulas o protótipo não consegue converter e que melhorias seriam necessárias.

Referências

ABELSON, H.; SUSSMAN, G. J. Structure and Interpretation of Computer Programs. [S.l.]: MIT Press, 1996.

Epic Games. Blueprint Visual Scripting. 2025. Acesso em: 15 jun. 2026. Disponível em: <https://dev.epicgames.com/documentation/unreal-engine/blueprints-visual-scripting-in-unreal-engine>.

GIL, A. C. Como elaborar projetos de pesquisa. 4. ed. São Paulo: Atlas, 2002.

KNUTH, D. E. The Art of Computer Programming. [S.l.]: Addison-Wesley, 1968. v. 1.

Microsoft. Work with triggers and actions — Power Automate. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://learn.microsoft.com/en-us/power-automate/work-with-triggers-actions>.

MYERS, B. A. Taxonomies of visual programming and program visualization. Journal of Visual Languages and Computing, v. 1, n. 1, p. 97–123, 1990.

n8n. Nodes — n8n Documentation. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://docs.n8n.io/workflows/components/nodes/>.

NOONE, M.; MOONEY, A. Visual and textual programming languages: A systematic review of the literature. Journal of Computers in Education, v. 5, n. 2, p. 149–174, 2018.

OpenJS Foundation. Node-RED: Low-code programming for event-driven applications. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://nodered.org/>.

PAN, R. et al. Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24). Lisboa, Portugal: [s.n.], 2024.

PRODANOV, C. C.; FREITAS, E. C. d. Metodologia do trabalho científico. 2. ed. Novo Hamburgo: Feevale, 2013.

ROMERO, M. A Linguagem Visual Blueprints da Unreal Engine 4. 2014. Acesso em: 15 jun. 2026. Disponível em: <https://romeroblueprints.blogspot.com/2014/03/a-linguagem-visual-blueprints-da-unreal.html>.

SÁEZ-LÓPEZ, J.-M.; ROMÁN-GONZÁLEZ, M.; VÁZQUEZ-CANO, E. Visual programming languages integrated across the curriculum in elementary school: A two year case study using "scratch" in five schools. Computers & Education, v. 97, p. 129–141, 2016.

SILVA, J. X. et al. Low-code and no-code technologies adoption: A gray literature review. In: Anais do XIX Simpósio Brasileiro de Sistemas de Informação. Porto Alegre: Sociedade Brasileira de Computação, 2023.

WAYAMA, K. et al. Verifying game logic in unreal engine 5 blueprint visual scripting system using model checking. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE). [S.l.: s.n.], 2022.